

Common Mistakes in Optimization: A Self-Diagnosis Guide

CS 182: Deep Neural Networks

Abstract

This guide helps you identify and correct common misconceptions about optimization in deep learning. Rather than memorizing corrections, you'll develop a geometric intuition for how data structure—particularly the singular value spectrum—drives optimization behavior. Use this as a self-diagnostic tool when working through problems.

1 Introduction: The Central Insight

Most mistakes in understanding optimization stem from ignoring a fundamental principle: **optimization behavior is determined by the geometry of your data, not just its size.** Specifically, the singular value decomposition (SVD) of your data matrix $\mathbf{X}$ reveals how fast gradient descent converges in different directions, which directions Ridge regression suppresses, and why early stopping works as implicit regularization.

Think of it this way: your data matrix $\mathbf{X}$ defines a coordinate system with axes given by the singular vectors $\vec{v}_i$, and the singular values σ_i tell you how much "stretch" there is along each axis. Optimization algorithms respond to this geometry in predictable, analyzable ways.

2 Theme 1: Spectral Properties vs. Dataset Size

2.1 Mistake 1.1: Choosing Learning Rate Based on Dataset Size

How to Spot This in Your Own Thinking

Warning Signs:

- You write something like: "Since $n = 10,000$ is large, I'll use a small learning rate like $\eta = 0.0001$ "
- You increase the learning rate when working with a smaller dataset
- You don't consider the actual values in $\mathbf{X}$ when setting η
- Your justification for η mentions only n or d , not properties like $\sigma_{\max}$

Why this feels plausible: Intuitively, more data seems like it should require more careful, smaller steps. Also, many online tutorials mention "large datasets need small learning rates" without explaining why.

What's Actually Happening

The convergence condition for gradient descent on least squares is:

$$\eta < \frac{1}{\lambda_{\max}(\mathbf{X}^T \mathbf{X})} = \frac{1}{\sigma_{\max}^2(\mathbf{X})} \quad (1)$$

Notice that n (number of samples) and d (number of features) don't appear in this bound! What matters is $\sigma_{\max}$, the largest singular value. Here's the geometric intuition:

- $\sigma_{\max}$ measures the maximum "stretch" of the data. If your data points lie in a long, thin ellipsoid, $\sigma_{\max}$ is large.
- A large $\sigma_{\max}$ means the loss surface has steep curvature in some direction.
- If you take too large a step in a steep direction, you overshoot and oscillate.

Worked Example: Consider two datasets, both with $n = 1000$ samples:

Dataset A: $\mathbf{X}_A \in \mathbb{R}^{1000 \times 2}$ where all entries are drawn from $\mathcal{N}(0, 1)$. Expected $\sigma_{\max} \approx \sqrt{1000} \approx 31.6$, so $\eta < 1/1000 \approx 0.001$.

Dataset B: $\mathbf{X}_B \in \mathbb{R}^{1000 \times 2}$ where all entries are drawn from $\mathcal{N}(0, 100)$. Expected $\sigma_{\max} \approx 100\sqrt{1000} \approx 3162$, so $\eta < 1/(10^7) = 10^{-7}$.

Both have $n = 1000$, but Dataset B requires a learning rate 10,000 times smaller! The difference is the scale and geometry of the data, not the dataset size.

Complete 2×2 Example

Let $\mathbf{X} = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}$. This is already in SVD form: $\mathbf{U} = \mathbf{V} = \mathbf{I}$, and $\mathbf{\Sigma} = \mathbf{X}$ with $\sigma_1 = 10$, $\sigma_2 = 1$.

The convergence condition is $\eta < 1/\sigma_1^2 = 1/100 = 0.01$.

Let's test $\eta = 0.02$ (too large) on the problem $\mathbf{X}\vec{w} = \begin{bmatrix} 10 \\ 1 \end{bmatrix}$ starting from $\vec{w}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$:

$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 100 & 0 \\ 0 & 1 \end{bmatrix}$, so the update is:

$$\vec{w}_1 = \vec{w}_0 - 2\eta \mathbf{X}^T (\mathbf{X}\vec{w}_0 - \vec{y}) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.04 \begin{bmatrix} 100 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} -10 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ 0.04 \end{bmatrix}$$

The true solution is $\vec{w}_* = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Notice $w_{1,1} = 4$ already overshoot! Continuing:

$$\begin{aligned} \mathbf{X}\vec{w}_1 - \vec{y} &= \begin{bmatrix} 40 \\ 0.04 \end{bmatrix} - \begin{bmatrix} 10 \\ 1 \end{bmatrix} = \begin{bmatrix} 30 \\ -0.96 \end{bmatrix} \\ \vec{w}_2 &= \begin{bmatrix} 4 \\ 0.04 \end{bmatrix} - 0.04 \begin{bmatrix} 100 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 30 \\ -0.96 \end{bmatrix} = \begin{bmatrix} -8 \\ 0.078 \end{bmatrix} \end{aligned}$$

The first component oscillates wildly ($0 \rightarrow 4 \rightarrow -8 \rightarrow \dots$) while the second component converges smoothly. The first direction corresponds to $\sigma_1 = 10$, which violates our learning rate bound.

Self-Check Questions

1. If I double all entries in my data matrix $\mathbf{X}$, how should I adjust my learning rate?
2. Can two datasets with the same dimensions ($n \times d$) require vastly different learning rates? Give an example.
3. If I standardize my features (zero mean, unit variance), how does this affect $\sigma_{\max}$ and thus my choice of η ?
4. **Computational check:** Take a random 10×5 matrix. Compute $\sigma_{\max}$ using SVD. Run GD with $\eta = 1/\sigma_{\max}^2$ and $\eta = 2/\sigma_{\max}^2$. Does the second one oscillate?

Connection to Practice

In neural networks, adaptive optimizers like Adam effectively compute per-parameter learning rates based on gradient statistics, which implicitly account for the local geometry. However, understanding the $\sigma_{\max}$ principle remains crucial:

- When features have vastly different scales, vanilla GD fails catastrophically—this is why feature normalization is standard.
- The principle extends to neural networks: the effective learning rate interacts with the curvature of the loss landscape.
- In practice, $\sigma_{\max}$ is rarely computed directly, but gradient norm clipping and layer normalization are techniques that implicitly control spectral properties.

3 Theme 2: Directional Behavior in Optimization

3.1 Mistake 2.1: Expecting Uniform Convergence Across All Directions

How to Spot This in Your Own Thinking

Warning Signs:

- You're surprised that some parameters converge quickly while others barely move
- You think one learning rate should work equally well for all parameters
- You don't understand why the condition number $\kappa = \sigma_{\max}/\sigma_{\min}$ matters
- You expect the error to decrease uniformly across iterations

Why this feels plausible: GD is a systematic algorithm applying the same rule everywhere, so it seems like it should make uniform progress.

What's Actually Happening

Recall the GD update in the SVD basis. For component i :

$$\tilde{w}_{t+1,i} = (1 - 2\eta\sigma_i^2)\tilde{w}_{t,i} + 2\eta\sigma_i\tilde{y}_i \quad (2)$$

The key insight: **the effective step size in direction i is $2\eta\sigma_i^2$, not η .**

- For large σ_i : GD takes large effective steps, rapid convergence
- For small σ_i : GD takes tiny effective steps, very slow convergence
- The convergence rate differs by a factor of $\sigma_{\max}^2/\sigma_{\min}^2 = \kappa^2$

Geometric picture: Imagine the loss surface as an elongated bowl. GD quickly descends the steep walls (large σ_i directions) but takes forever to navigate the flat valley floor (small σ_i directions).

Worked Example with Iterations

Consider $\mathbf{X} = \begin{bmatrix} 3 & 0 \\ 0 & 0.3 \end{bmatrix}$ with $\sigma_1 = 3$, $\sigma_2 = 0.3$ (condition number $\kappa = 10$).

Solve $\mathbf{X}\vec{w} = \begin{bmatrix} 3 \\ 0.3 \end{bmatrix}$ with $\vec{w}_0 = \vec{0}$ and $\eta = 0.1$ (safe: $0.1 < 1/9$).

True solution: $\vec{w}_* = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Update rule: $\vec{w}_{t+1} = (I - 2\eta\mathbf{X}^T\mathbf{X})\vec{w}_t + 2\eta\mathbf{X}^T\vec{y}$

With $\mathbf{X}^T\mathbf{X} = \begin{bmatrix} 9 & 0 \\ 0 & 0.09 \end{bmatrix}$:

$$\begin{aligned} t = 0 : \quad \vec{w}_0 &= \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \text{Error: } \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ t = 1 : \quad \vec{w}_1 &= \begin{bmatrix} 0.6 \\ 0.06 \end{bmatrix}, \quad \text{Error: } \begin{bmatrix} 0.4 \\ 0.94 \end{bmatrix} \\ t = 2 : \quad \vec{w}_2 &= \begin{bmatrix} 0.84 \\ 0.116 \end{bmatrix}, \quad \text{Error: } \begin{bmatrix} 0.16 \\ 0.884 \end{bmatrix} \\ t = 5 : \quad \vec{w}_5 &= \begin{bmatrix} 0.990 \\ 0.263 \end{bmatrix}, \quad \text{Error: } \begin{bmatrix} 0.010 \\ 0.737 \end{bmatrix} \end{aligned}$$

After 5 iterations, the first component has 99% of the error eliminated, while the second component still has 74% remaining! This is because:

- Convergence rate in direction 1: $(1 - 2 \cdot 0.1 \cdot 9) = -0.8$ (magnitude 0.8, so error multiplied by 0.8 per iteration)
- Convergence rate in direction 2: $(1 - 2 \cdot 0.1 \cdot 0.09) = 0.982$ (error multiplied by 0.982 per iteration)

The ratio of convergence speeds is $(0.982/0.8) \approx 1.23$ per iteration, which compounds over time.

Self-Check Questions

1. If $\sigma_1 = 100\sigma_2$, roughly how many more iterations does it take to reduce error in direction 2 compared to direction 1?
2. Why is it impossible to choose η such that all directions converge at the same rate?
3. How does the condition number κ relate to the "flatness" of the loss surface?
4. **Computational check:** Create a diagonal matrix with diagonal entries $[10, 1, 0.1]$. Run GD and plot error in each direction vs. iteration. Does it match your prediction?

Connection to Practice

This directional behavior is fundamental:

- **Preconditioning:** Advanced optimizers (e.g., Adam, RMSprop) try to equalize convergence rates across directions by rescaling updates.
- **Batch normalization:** Helps by controlling the effective condition number of activations.
- **When it matters:** For well-conditioned problems ($\kappa \approx 1$), vanilla GD works fine. For ill-conditioned problems ($\kappa \gg 1$), you need adaptive methods or momentum.

3.2 Mistake 2.2: Thinking Ridge Regression Shrinks All Directions Uniformly

How to Spot This in Your Own Thinking

Warning Signs:

- You describe Ridge as "making all weights smaller"
- You expect $\vec{w}_{\text{ridge}}$ to be a scaled version of $\vec{w}_{\text{OLS}}$
- You don't consider which directions in parameter space are affected most
- You think larger λ always means smaller $\|\vec{w}\|$

What's Actually Happening

The Ridge solution in the SVD basis is:

$$\vec{w}_* = \sum_{i=1}^{\text{rank}(\mathbf{X})} \vec{v}_i \left(\frac{\sigma_i}{\sigma_i^2 + \lambda} \right) (\vec{u}_i^T \vec{y}) \quad (3)$$

The scaling factor for direction i is $\frac{\sigma_i}{\sigma_i^2 + \lambda}$. Let's analyze this:

- If $\sigma_i^2 \gg \lambda$: scaling $\approx \frac{\sigma_i}{\sigma_i^2} = \frac{1}{\sigma_i}$ (nearly unaffected)
- If $\sigma_i^2 \ll \lambda$: scaling $\approx \frac{\sigma_i}{\lambda} \approx 0$ (strongly suppressed)
- If $\sigma_i^2 \approx \lambda$: scaling $\approx \frac{1}{2\sigma_i}$ (intermediate suppression)

Key insight: Ridge regularization **selectively** suppresses low-variance directions (small σ_i), which often correspond to noise. High-variance directions (large σ_i) are relatively unaffected.

Worked Example

Consider $\mathbf{X} = \begin{bmatrix} 2 & 0 \\ 0 & 0.2 \end{bmatrix}$ with $\vec{y} = \begin{bmatrix} 4 \\ 0.4 \end{bmatrix}$, so $\sigma_1 = 2$, $\sigma_2 = 0.2$.

OLS solution: $\vec{w}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \vec{y} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

Ridge solution with $\lambda = 0.1$:

Scaling factor 1: $\frac{\sigma_1}{\sigma_1^2 + \lambda} = \frac{2}{4 + 0.1} = \frac{2}{4.1} \approx 0.488$

Scaling factor 2: $\frac{\sigma_2}{\sigma_2^2 + \lambda} = \frac{0.2}{0.04 + 0.1} = \frac{0.2}{0.14} \approx 1.429$

Wait, this seems wrong! Let's compute properly. The SVD form gives us:

$$\vec{w}_{\text{Ridge}} = \sum_i \vec{v}_i \left(\frac{\sigma_i}{\sigma_i^2 + \lambda} \right) (\vec{u}_i^T \vec{y})$$

For our diagonal case: $\vec{u}_1^T \vec{y} = 4$, $\vec{u}_2^T \vec{y} = 0.4$, and $\vec{v}_i = \vec{e}_i$.

$$w_{1,\text{Ridge}} = \frac{2}{4.1} \cdot 4 \approx 1.95 \quad (\text{OLS: } 2, \text{ shrinkage: } 2.5\%)$$

$$w_{2,\text{Ridge}} = \frac{0.2}{0.14} \cdot 0.4 \approx 0.57 \quad (\text{OLS: } 2, \text{ shrinkage: } 71.5\%)$$

The second component (small σ_2) is shrunk by 71.5%, while the first (large σ_1) is barely affected (only 2.5% shrinkage)!

Self-Check Questions

1. For which value of σ_i does Ridge have the strongest relative effect: $\sigma_i = 10$, $\sigma_i = 1$, or $\sigma_i = 0.1$ (assuming fixed $\lambda = 1$)?
2. If all singular values are very large compared to λ , what does Ridge regression approximately equal?
3. Why does suppressing low-variance directions help prevent overfitting?
4. **Computational check:** For the example above, compute Ridge solutions with $\lambda \in \{0.01, 0.1, 1, 10\}$. At what λ does the shrinkage become approximately equal for both directions?

Connection to Practice

- Low-variance directions often capture noise rather than signal, so suppressing them improves generalization.
- The choice of λ determines the cutoff: directions with $\sigma_i^2 < \lambda$ are strongly regularized.
- In deep learning, weight decay (L2 regularization) serves a similar role, though the geometry is more complex.

4 Theme 3: Regularization Across Time and Space

4.1 Mistake 3.1: Missing the Connection Between Early Stopping and Ridge

How to Spot This in Your Own Thinking

Warning Signs:

- You think early stopping and Ridge are unrelated techniques
- You don't see why stopping GD before convergence would help generalization
- You believe you must always run optimization until full convergence
- You can't explain what "implicit regularization" means

What's Actually Happening

Recall that GD converges at different rates in different directions:

$$\tilde{w}_{t,i} \approx \tilde{w}_{*,i}(1 - (1 - 2\eta\sigma_i^2)^t) \quad (4)$$

For large σ_i : $(1 - 2\eta\sigma_i^2)^t \rightarrow 0$ rapidly, so $\tilde{w}_{t,i} \rightarrow \tilde{w}_{*,i}$ fast.

For small σ_i : $(1 - 2\eta\sigma_i^2)^t$ decays slowly, so $\tilde{w}_{t,i}$ remains small.

The connection: If you stop at iteration T before full convergence:

- High-variance directions (large σ_i) have converged
- Low-variance directions (small σ_i) haven't converged yet—they're still suppressed

This is exactly what Ridge does! Both prevent fitting to low-variance directions. The difference:

- Ridge: suppression via λ (space)
- Early stopping: suppression via limited iterations T (time)

Approximate relationship: Early stopping at iteration T with learning rate η is roughly equivalent to Ridge with $\lambda \approx 1/(\eta T)$.

Worked Example

Using the same setup as before: $\mathbf{X} = \begin{bmatrix} 3 & 0 \\ 0 & 0.3 \end{bmatrix}$, $\vec{y} = \begin{bmatrix} 3 \\ 0.3 \end{bmatrix}$, $\eta = 0.1$.

Suppose we stop at $T = 3$ iterations:

$$\vec{w}_3 \approx \begin{bmatrix} 0.936 \\ 0.174 \end{bmatrix} \quad (\text{from earlier calculation})$$

Compare to Ridge with $\lambda = 1/(\eta T) = 1/0.3 \approx 3.33$:

$$w_{1,\text{Ridge}} = \frac{9}{9 + 3.33} \cdot 1 \approx 0.730$$

$$w_{2,\text{Ridge}} = \frac{0.09}{0.09 + 3.33} \cdot 1 \approx 0.026$$

The patterns are similar: both suppress the small singular value direction more than the large one, though the exact values differ due to the approximate nature of the relationship.

Self-Check Questions

1. Why does early stopping work better when the condition number κ is large?
2. If you double the number of iterations before stopping, how should you adjust λ to get a similar Ridge solution?
3. Can early stopping cause underfitting? When?
4. **Computational check:** Run GD for 5, 10, 20, 50 iterations. For each, compute the corresponding Ridge λ and verify they give similar solutions.

Connection to Practice

- Early stopping is one of the most practical regularization techniques in deep learning.
- Validation error typically reaches a minimum before training error, suggesting when to stop.
- This is why SGD with many epochs on complex models doesn't necessarily overfit—the noise in SGD provides additional implicit regularization.

4.2 Mistake 3.2: Believing SGD Gradients Are Biased

How to Spot This in Your Own Thinking

Warning Signs:

- You say "SGD uses approximate gradients" and think this means systematic error
- You confuse variance with bias
- You think SGD "can't find the true optimum" because of noisy gradients
- You believe mini-batch gradients are always inferior to full-batch gradients

What's Actually Happening

The crucial property is:

$$\mathbb{E}_{i \sim \text{Uniform}}[\nabla f_i(\theta)] = \nabla f(\theta) \quad (5)$$

This means the mini-batch gradient is an **unbiased estimator** of the true gradient. Let's be precise:

- **Bias:** $\mathbb{E}[\hat{\theta}] - \theta$. For SGD gradients, bias = 0.
- **Variance:** $\mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]$. SGD gradients have high variance.

High variance means individual gradient estimates are noisy, but they're noisy in an unbiased way—they don't systematically point in the wrong direction. Over many iterations, the errors average out.

Numerical Illustration

Consider $f(w) = w^2$ with data points giving gradients:

$$\nabla f_1(1) = 3, \quad \nabla f_2(1) = 1, \quad \nabla f_3(1) = 2, \quad \nabla f_4(1) = 2$$

True gradient: $\nabla f(1) = 2$ (average).

If we sample $i = 1$, we get $\nabla f_1(1) = 3$ (error: +1). If we sample $i = 2$, we get $\nabla f_2(1) = 1$ (error: -1).

Over many samples: $\mathbb{E}[\text{error}] = \frac{1}{4}(1 + (-1) + 0 + 0) = 0$. **Unbiased!**

But $\text{Var} = \mathbb{E}[(\text{error})^2] = \frac{1}{4}(1 + 1 + 0 + 0) = 0.5 > 0$. **High variance!**

Self-Check Questions

1. If you average the mini-batch gradients from 1000 iterations of SGD, what should you get (approximately)?
2. Does increasing batch size reduce bias, variance, or both?
3. Why might high variance actually be beneficial for non-convex optimization?
4. **Computational check:** For a simple quadratic, compute gradients on 100 random mini-batches. Verify their average equals the full gradient.

Connection to Practice

- The variance decreases with larger batch sizes: $\text{Var}[\text{mini-batch gradient}] \propto 1/|\mathcal{B}|$.
- In deep learning, moderate batch sizes (32-256) work well: small enough for fast iterations, large enough for reasonable variance.
- The noise in SGD can help escape sharp minima, leading to flatter minima that generalize better.

5 Theme 4: Understanding Momentum

5.1 Mistake 4.1: Thinking Momentum Is Just "Bigger Steps"

How to Spot This in Your Own Thinking

Warning Signs:

- You describe momentum as "taking larger steps"
- You think momentum and increasing η are equivalent
- You don't understand why momentum helps with oscillations
- You believe momentum only accelerates convergence, not smooths it

What's Actually Happening

The momentum update is:

$$\vec{z}_{k+1} = \beta \vec{z}_k + (1 - \beta) \nabla f(\vec{w}_k) \quad (6)$$

$$\vec{w}_{k+1} = \vec{w}_k - \eta \vec{z}_{k+1} \quad (7)$$

Expanding the recurrence:

$$\vec{z}_k = (1 - \beta) \sum_{i=0}^{k-1} \beta^{k-1-i} \nabla f(\vec{w}_i) \quad (8)$$

This is an **exponentially weighted moving average** of past gradients, not just a bigger step. Key properties:

- Recent gradients get weight $(1 - \beta)\beta^0 = (1 - \beta)$
- Gradients from t steps ago get weight $(1 - \beta)\beta^t$ (exponentially decaying)
- The "effective memory" is approximately $1/(1 - \beta)$ steps

Why this helps with oscillations: If the gradient oscillates (e.g., $+1, -1, +1, -1, \dots$), the moving average smooths these out. If the gradient is consistent in one direction, momentum builds up velocity.

Think of it as a ball rolling down a hill with friction: it accelerates in consistent directions but doesn't jitter in oscillating directions.

Numerical Example

Consider GD with oscillating gradients: $\nabla f(\vec{w}_0) = 10, \nabla f(\vec{w}_1) = -8, \nabla f(\vec{w}_2) = 10, \nabla f(\vec{w}_3) = -8$.

Without momentum ($\eta = 0.1$):

$$w_1 = w_0 - 0.1 \cdot 10 = w_0 - 1$$

$$w_2 = w_1 - 0.1 \cdot (-8) = w_0 - 1 + 0.8 = w_0 - 0.2$$

$$w_3 = w_2 - 0.1 \cdot 10 = w_0 - 1.2$$

$$w_4 = w_3 - 0.1 \cdot (-8) = w_0 - 0.4$$

Net progress after 4 steps: 0.4 (lots of back-and-forth).

With momentum ($\beta = 0.9, \eta = 0.1$):

$$\begin{aligned}z_1 &= 0.1 \cdot 10 = 1 \\w_1 &= w_0 - 0.1 \cdot 1 = w_0 - 0.1 \\z_2 &= 0.9 \cdot 1 + 0.1 \cdot (-8) = 0.9 - 0.8 = 0.1 \\w_2 &= w_1 - 0.1 \cdot 0.1 = w_0 - 0.11 \\z_3 &= 0.9 \cdot 0.1 + 0.1 \cdot 10 = 1.09 \\w_3 &= w_2 - 0.1 \cdot 1.09 = w_0 - 0.22 \\z_4 &= 0.9 \cdot 1.09 + 0.1 \cdot (-8) = 0.181 \\w_4 &= w_3 - 0.1 \cdot 0.181 = w_0 - 0.24\end{aligned}$$

Net progress after 4 steps: 0.24 with much smoother trajectory.

Self-Check Questions

1. If gradients are consistent (always pointing the same direction), how does momentum help?
2. What's the relationship between β and the "memory length" of momentum?
3. Can momentum cause overshooting? Under what conditions?
4. **Computational check:** Implement momentum from scratch. Verify that with $\beta = 0$ you recover standard GD.

Connection to Practice

- Typical values: $\beta \in [0.9, 0.99]$ in deep learning.
- Momentum especially helps with ill-conditioned problems (large κ).
- Modern optimizers (Adam) combine momentum with adaptive learning rates for best of both worlds.

6 Quick Reference & Debugging Checklist

Key Formulas with Geometric Interpretations:

- **GD convergence:** $\eta < 1/\sigma_{\max}^2$
The learning rate is limited by the steepest direction, not dataset size.
- **Ridge solution:** $\vec{w}_* = \sum_i \vec{v}_i \frac{\sigma_i}{\sigma_i^2 + \lambda} (\vec{u}_i^T \vec{y})$
Suppresses components with $\sigma_i^2 < \lambda$; leaves large σ_i nearly unchanged.
- **GD in SVD basis:** $\tilde{w}_{t+1,i} = (1 - 2\eta\sigma_i^2)\tilde{w}_{t,i} + 2\eta\sigma_i\tilde{y}_i$
Effective step size is $2\eta\sigma_i^2$ —large σ_i converge fast, small σ_i converge slowly.
- **Condition number:** $\kappa = \sigma_{\max}/\sigma_{\min}$
Measures elongation of loss surface; larger κ means slower convergence.
- **SGD gradient:** $\mathbb{E}[\nabla f_i(\theta)] = \nabla f(\theta)$
Unbiased but high variance; variance $\propto 1/|\mathcal{B}|$.

Debugging Checklist for Problem Sets:

1. **Before choosing η :** Did I compute or estimate $\sigma_{\max}$? Did I check if features have vastly different scales?
2. **If GD oscillates:** Is η too large? Specifically, is $\eta > 1/\sigma_{\max}^2$?
3. **If GD converges very slowly:** Is the condition number κ large? Should I consider momentum or preconditioning?
4. **When analyzing Ridge:** Am I thinking about which directions (large vs. small σ_i) are affected most?
5. **For early stopping:** Have I identified which directions converge first? Do I understand why stopping early prevents overfitting?
6. **For SGD:** Am I confusing bias and variance? Do I understand why unbiased noisy estimates still converge?
7. **For momentum:** Do I see it as exponential averaging, not just bigger steps?

7 Experiments to Deepen Understanding

Try these computational experiments to build intuition:

1. **Spectral sensitivity:** Create two 2×2 matrices with the same Frobenius norm but different condition numbers. Run GD on both and observe convergence speed.
2. **Ridge vs. early stopping:** For a fixed problem, find the λ that gives a Ridge solution closest to GD stopped at iteration $T = 10$. Does the relationship $\lambda \approx 1/(\eta T)$ hold?
3. **SGD bias:** Run SGD with batch size 1 for 1000 iterations. Average all gradient estimates. How close is it to the true gradient?
4. **Momentum smoothing:** Generate synthetic oscillating gradients. Plot the trajectory with and without momentum. Measure the "smoothness" (variance of step directions).

8 Closing Thoughts

Understanding optimization isn't about memorizing convergence rates—it's about developing geometric intuition for how algorithms navigate loss landscapes. The singular value decomposition provides a powerful lens: it reveals the "natural coordinates" of your problem, showing you which directions are easy or hard for optimization.

When you encounter a mistake, ask yourself: *Am I thinking about the geometry of my data, or just the size?* This question alone will resolve most misconceptions.

Keep experimenting, keep visualizing, and remember: every "mistake" is an opportunity to refine your mental model. The goal isn't to never make errors, but to develop the skills to diagnose and correct them independently.